

PRIORITY QUEUES

A priority queue is just like a normal queue, but each element has a priority.

- In a normal queue → First Come, First Serve (like a line at the ticket counter).
- In a priority queue → The element with **higher priority** comes out first, not necessarily the one that came in first.
- Elements with higher priority are typically retrieved or removed before elements with less priority.
- Based on priority, types of Priority Queues are – Minimum Priority Queue and Maximum Priority Queue.
- **Maximum Priority Queue** – The element with the highest value is considered the highest priority element.
- **Minimum Priority Queue** – The element with the minimum priority is considered the highest priority element.
- There are different ways to implement a Priority Queue. We can implement priority queue using Arrays, Linked List, Binary Trees, Binary Search Trees.

PRIORITY QUEUE



REMOVAL ORDER

TASK C → TASK A → TASK B

Operations in Priority Queue:

- **Insert** – inserts an element into the priority queue, **Get** - retrieves the element with has the highest priority, **Remove** – Removes the element with the highest priority in the priority queue.
- Time complexity Analysis for implementing a Priority Queue.

Data Structure / Operation	Insertion of element	Get Minimum Element	Remove the Minimum Element
Unsorted Array	$O(1)$	$O(N)$	$O(N)$
Sorted Array	$O(N)$	$O(1)$	$O(1)$ – Finding the Element $O(N)$ – Removing the Element
Unsorted Linked List	$O(1)$	$O(N)$	$O(N)$
Sorted Linked List	$O(N)$	$O(1)$	$O(1)$
Binary Search Tree	$O(H)$	$O(H)$	$O(H)$
Balanced Binary Search Tree	$O(\log N)$	$O(\log N)$	$O(\log N)$
HashMap	$O(1)$	$O(N)$	$O(N)$
Heaps	$O(\log N)$	$O(\log N)$	$O(\log N)$

H – Height of the Tree

N – Number of elements in Priority Queue.

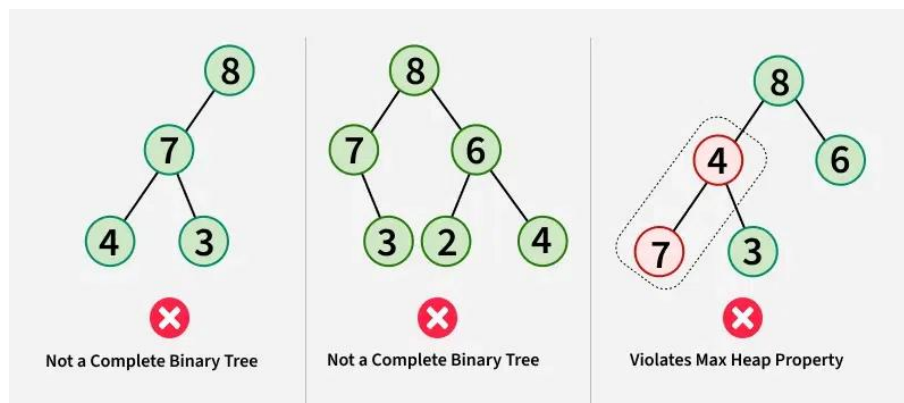
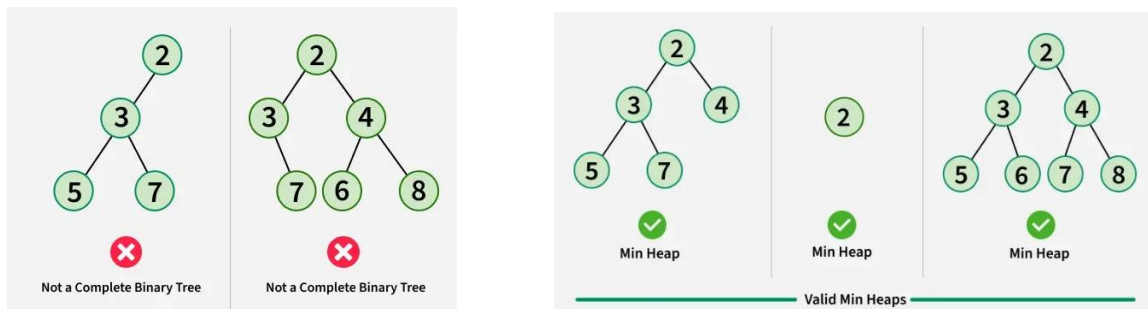
Heaps:

A **heap** is a special type of **binary tree** (a tree where each parent has at most 2 children). It is mainly used to **implement priority queues** efficiently. In order to implement Heap, follows 2 properties.

1. **Complete Binary Tree** - All levels are **completely filled**, except the last. In last level, nodes are filled from **left to right**.

2. Heap Order Property:

- In a **Min Heap** → Every parent is **smaller** than (or equal to) its children.
- In a **Max Heap** → Every parent is **greater** than (or equal to) its children.

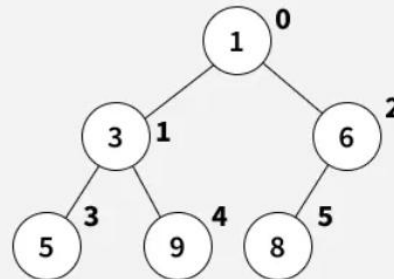


A heap is often represented as an array rather than a tree structure. In this array-based representation, for any element at index i , the left child is at index $2i + 1$ and the right child is at index $2i + 2$. The parent of any element at index i is at index $(i-1)/2$

actual representation



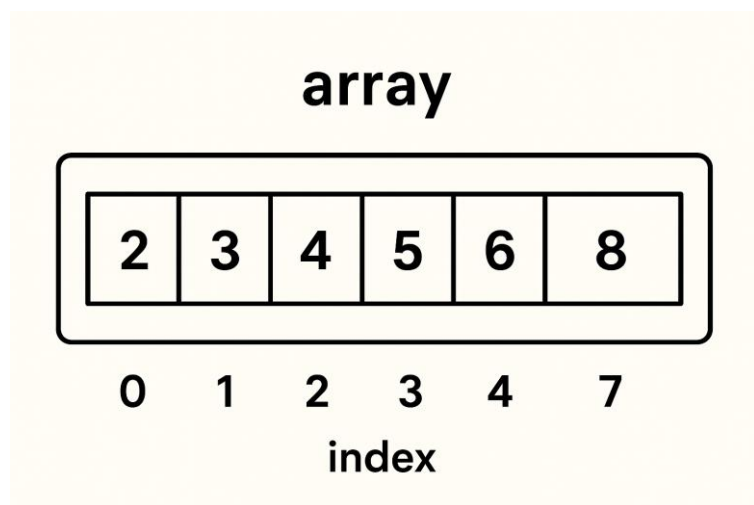
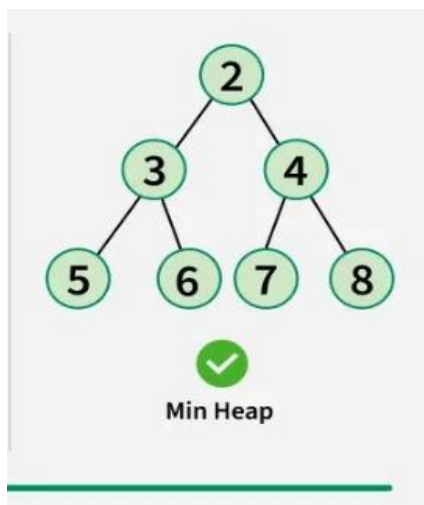
pictorial representation



Representation of a Binary Heap

Visualization of Heaps:

Heap is a **Complete Binary Tree**. A binary heap is typically represented as an array. Root element will be at `array[0]`.

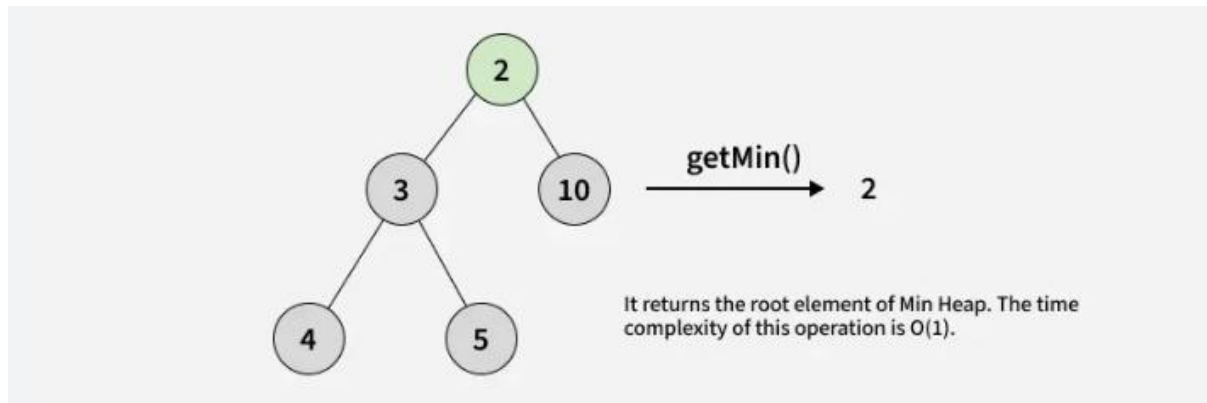


<code>arr[(i-1)/2]</code>	Returns the parent node	Element 2	Parent node
<code>arr[(2*i)+1]</code>	Returns the left child node	Element 3	Left child of Element 2
<code>arr[(2*i)+2]</code>	Returns the right child node	Element 4	Right child of Element 3.

OPERATIONS IN QUEUE

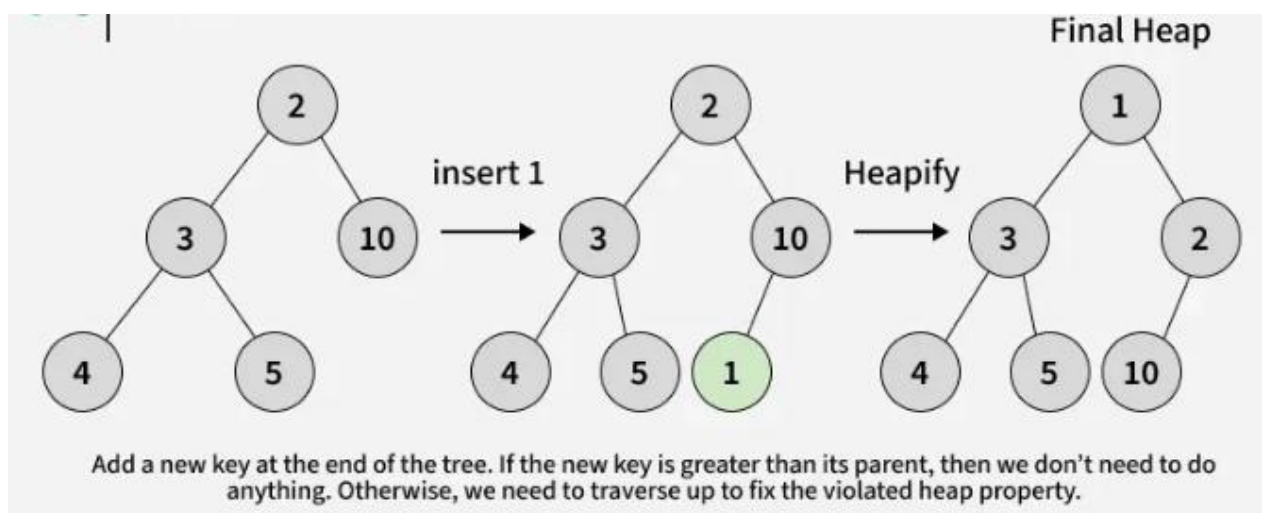
1. Get Minimum:

Gives the element with highest priority in the priority queue.



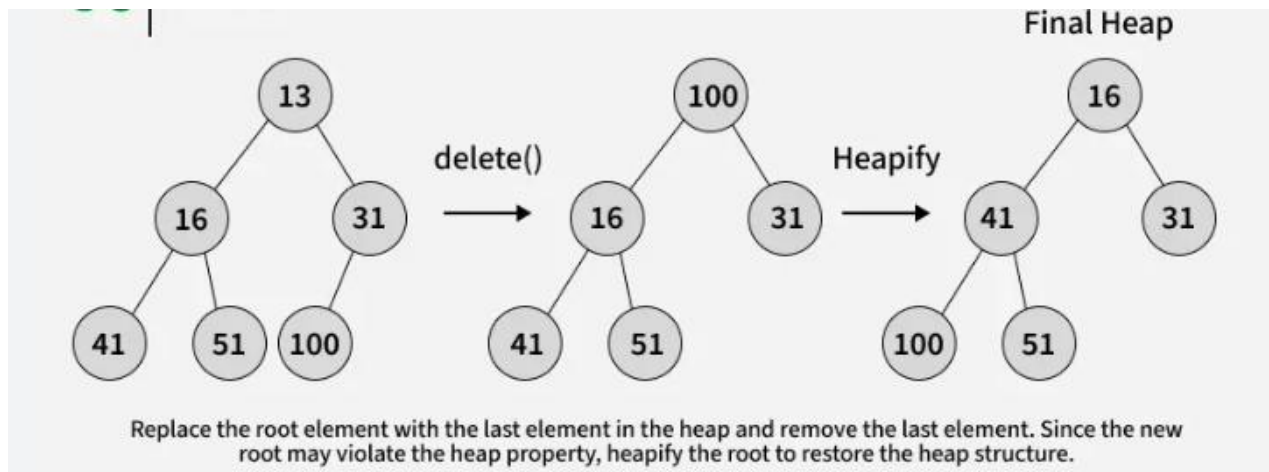
2. Insert into the Heap:

Inserts the element into the Heap and checks for priority in-order to maintain the Heap order property.



3. Remove an element in Heap:

Removes the element with the Highest priority, and replace root with a node, which maintains the Heap order property.



QUEUE VS PRIORITY QUEUE:

Feature	Queue	Priority Queue
Order of Removal	FIFO (First-In-First-Out)	Based on Priority
Example	Line at a ticket counter	Emergency room patients
Implementation	Array / Linked List	Heap / BST / Arrays / Lists

Pseudocode – Min Heap:

- **Root (top element):** Always the **smallest** element.
- **Children:** Every parent node is **smaller than or equal to** its children.

Insert Operation:

```
function insert (element):  
    put the element at the end of the heap  
    while element is smaller than its parent:  
        swap element with its parent
```

Get Minimum Operation:

```
function getMin ():  
    return the element at the root (first element)
```

Remove Minimum Operation:

```
function removeMin ():  
    save the root element (this is the minimum)  
    move the last element to the root  
    remove the last element  
    while root is bigger than any of its children:  
        swap root with the smaller child  
    return the saved minimum
```

Pseudocode – Max Heap:

- **Root (top element):** Always the largest element.
- **Children:** Every parent node is **greater than or equal** to its children.

Insert Operation:

```
function insert (element):  
    put the element at the end of the heap  
    while element is greater than its parent:  
        swap element with its parent
```

Get Minimum Operation:

```
function getMax ():  
    return the element at the root (first element)
```

Remove Maximum Operation:

```
function removeMax ():  
    save the root element (this is the maximum)  
    move the last element to the root  
    remove the last element  
    while root is smaller than any of its children:  
        swap root with the smaller child  
    return the saved maximum
```


Real Time Applications of Priority Queue:

- **Operating Systems OS Scheduling-** CPU processes are given different priorities (e.g., system processes > user apps).
- **Hospital Emergency Room** - Patients with critical conditions get treated first, regardless of arrival time.
- **Network Routers & Bandwidth Management** - Packets are prioritized (e.g., video call packets > background file downloads).
- **Dijkstra's Algorithm & A Search (Graphs)** - Priority queues (min-heaps) are used to efficiently pick the next closest node.